

SelAction: open questions on the model

Addendum: two further points on the inputs
for Piter Bijma and Jack Dekkers

Austin Putz

6 October 2026

Abstract

This short addendum follows the open questions sent on 5 October 2026. While mapping every question the program asks and which of our tests reaches it, we found two further points we cannot settle from the code alone. Question 8 is new: the program does not accept a common-environmental effect of zero in the progeny test. The second point adds to Question 6: discrete and overlapping generations treat half-sib groups differently when each sire has one dam. Neither changes an equation, and we have changed nothing in the program.

1 Question 8: zero common-environmental effect in the progeny test

When common environmental effects are switched on (`initc = y`) and progeny groups are used, the program asks for a separate c^2 for the progeny test of each trait (`ccprog`). It rejects zero:

```
read *,ccprog(q)
if (ccprog(q).le.0) then
  print *," wrong input, com.env.effect must be higher than 0!"
```

The ordinary c^2 of each trait (`cc`) is checked with `cc(p).lt.0` instead, so zero is accepted there. The same check appears in all four routines: `sel1s`, `sel2s`, `sel3s` (`seldiscrete.f90`) and `ovlp` (`selovlp.f90`). It is also in the original code.

The value only enters as a variance,

$$\sigma_{c,\text{prog}}^2 = c_{\text{prog}}^2 \sigma_P^2, \quad \text{Cov}_{c,\text{prog}}(p, q) = r_c(p, q) \sigma_{c,\text{prog}}(p) \sigma_{c,\text{prog}}(q),$$

and is never divided by. When common environmental effects are switched off, the program itself sets $c_{\text{prog}}^2 = 0$ and uses progeny groups normally. So zero appears to be numerically safe. As it stands, a user who wants common environmental effects among full sibs but none in the progeny test (for example, progeny reared in many separate environments) cannot enter that, and must give a small positive value instead.

Question 8

Is there a reason the progeny-test c^2 must be strictly positive, or should zero be allowed, as it is for the ordinary c^2 ?

Our recommendation

Allow $0 \leq c_{\text{prog}}^2 < 1$, the same rule as for the ordinary c^2 , unless there is a modelling reason against it. This is an input check only; no equation changes.

2 Addition to Question 6: half-sib groups when each sire has one dam

Question 6 asked whether half-sib sources should be removed at every stage when $M_s = M_d$. Mapping the inputs showed that the two kinds of selection handle this case differently:

- **Discrete generations** (`sel1s`, `sel2s`, `sel3s`) always ask “use half-sib groups?”. If $M_s = M_d$, they then remove half-sib sources from the index with the note “half-sib information is thus not possible” (`note_matrat`). As described in Question 6, the removal only applies to the final-stage index.
- **Overlapping generations** (`ovlp`) ask about half-sib groups only if $M_s < M_d$. With $M_s \geq M_d$, the question is skipped, no half-sib group exists, and there is no note.

The overlapping-generation rule is the stricter of the two, and it also covers $M_s > M_d$. That case is not handled separately in discrete generations.

Addition to Question 6

Should both kinds of selection follow the same rule? If so, is it “no half-sib groups unless $M_s < M_d$ ” (as in overlapping generations), or ask and then remove with a note (as in discrete generations)?

Our recommendation

Use one rule for both, decided together with Question 6. Our planned input checker would then reject half-sib groups when $M_s \geq M_d$, with a clear message, instead of accepting them and dropping them later.

Source

`fortran/` at commit `4e2a245`. The complete list of inputs, with the condition under which each is asked, is in `tests/input_map/README.md`.